

# Building and Securing a REST API

MoMo SMS Data Processing System

|                    |                                    |
|--------------------|------------------------------------|
| <b>Team:</b>       | [Team name]                        |
| <b>Members:</b>    | [Member 1], [Member 2], [Member 3] |
| <b>Repository:</b> | [GitHub repository link]           |
| <b>Team sheet:</b> | [Team participation sheet link]    |

## 1. Introduction to API Security

---

An API gives other programs direct access to an application's data. The MoMo SMS dataset holds financial information: amounts, balances, names of senders and receivers, and transaction references. If anyone can call the API, anyone can read or change these records. API security relies on these principles:

- **Authentication** – confirm who is calling the API (here: HTTP Basic Auth on every endpoint).
- **Authorization** – decide what an authenticated client is allowed to do (e.g. read-only vs. admin).
- **Confidentiality & integrity** – protect data in transit with HTTPS/TLS so it cannot be read or modified.
- **Input validation** – reject malformed or malicious input. The API checks JSON syntax, required fields, numeric types and unknown fields, and returns 400 Bad Request on failure.
- **Least information in errors** – error messages say what was wrong with the request and never expose stack traces or internal details.

Our API is built with Python's standard library (`http.server`). The XML file is parsed into a list of JSON objects at start-up. Records are kept in a list and in a dictionary index (`id` → record) for fast lookups. All five CRUD endpoints first check the Authorization header. A missing or invalid header returns 401 Unauthorized with a `WWW-Authenticate` header. Credentials are read from environment variables rather than hard-coded in the handler. They are compared in constant time with `hmac.compare_digest` so that response timing does not reveal them.

### Data parsing

`dsa/parse_xml.py` reads every `<sms>` element in `modified_sms_v2.xml` with `xml.etree.ElementTree`. It uses regular expressions to classify each message body into one of nine transaction types (INCOMING\_MONEY, PAYMENT, TRANSFER, BANK\_DEPOSIT, AIRTIME, BUNDLES, DIRECT\_DEBIT, WITHDRAWAL, REVERSAL). It then extracts *amount*, *currency*, *sender*, *receiver*, *fee*, *balance*, *transaction reference* and *timestamp*. Non-transaction SMS (OTP codes, promotions, service notices) are skipped, so every JSON object has the same structure. Of the 1,691 SMS in the dataset, 1,657 were parsed as transactions and 34 were skipped. Every parsed record has an amount, sender, receiver and timestamp. Eight transfers have no balance because the SMS itself leaves it blank, so the field is `null`.

## 2. API Endpoints

---

All endpoints require Basic Auth. Full request and response examples are in `docs/api_docs.md`.

| Method | Endpoint           | Description                                   | Success | Errors        |
|--------|--------------------|-----------------------------------------------|---------|---------------|
| GET    | /transactions      | List all transactions; optional ?type=PAYMENT | 200     | 401           |
| GET    | /transactions/{id} | Get one transaction                           | 200     | 400, 401, 404 |
| POST   | /transactions      | Create; requires transaction_type, amount     | 201     | 400, 401      |
| PUT    | /transactions/{id} | Update the given fields                       | 200     | 400, 401, 404 |
| DELETE | /transactions/{id} | Delete; returns the deleted record            | 200     | 400, 401, 404 |

### Example – GET with authentication

```
curl -u admin:password123 http://127.0.0.1:8000/transactions/1

HTTP/1.0 200 OK
{
  "id": 1, "transaction_type": "INCOMING_MONEY", "amount": 2000, "currency": "RWF",
  "sender": "Jane Smith", "receiver": "Account Holder", "fee": 0, "balance": 2000,
  "transaction_ref": "76662021700", "timestamp": "2024-05-10T16:30:51", ...
}
```

### Example – wrong credentials

```
curl -i -u admin:wrongpass http://127.0.0.1:8000/transactions

HTTP/1.0 401 Unauthorized
WWW-Authenticate: Basic realm="MoMo API"
{ "error": "Unauthorized: invalid or missing credentials", "status": 401 }
```

### Example – POST

```
curl -u admin:password123 -X POST -H "Content-Type: application/json" \
  -d '{"transaction_type": "PAYMENT", "amount": 2500, "receiver": "Kigali Coffee Shop"}' \
  http://127.0.0.1:8000/transactions

HTTP/1.0 201 Created
Location: /transactions/1658
{ "id": 1658, "transaction_type": "PAYMENT", "amount": 2500, "receiver": "Kigali Coffee Shop",
  "currency": "RWF", "fee": 0 }
```

### Error codes

| Code | Meaning                                                         |
|------|-----------------------------------------------------------------|
| 400  | Invalid JSON, missing/invalid/unknown fields, or non-numeric id |
| 401  | Missing, malformed or wrong credentials                         |
| 404  | Transaction id not found, or unknown route                      |
| 405  | Method not allowed on this resource (e.g. DELETE /transactions) |

## Testing

We tested the endpoints manually with curl (`scripts/curl_tests.ps1`, screenshots in `screenshots/`). We also wrote an automated suite, `tests/test_api.py`, with 16 tests. It covers every CRUD endpoint, valid/missing/wrong/malformed credentials, validation errors, and 404/405 cases. All 16 tests pass.

## 3. DSA Comparison: Linear Search vs Dictionary Lookup

---

`dsa/search_compare.py` looks up **every** transaction id with both methods. It first checks that both return the same record. It then times them with `timeit`: on the first 20 parsed records (1,000 repetitions), on all 1,657 parsed records (10 repetitions), and on 10,000 synthetic records to show how each method scales.

| Records | Linear search (µs / lookup) | Dictionary lookup (µs / lookup) | Speedup |
|---------|-----------------------------|---------------------------------|---------|
| 20      | 1.05                        | 0.21                            | 5.1×    |
| 1,657   | 68.11                       | 0.14                            | 481×    |
| 10,000  | 439.12                      | 0.17                            | 2,594×  |

Linear search time grows roughly in proportion to the number of records. Dictionary lookup time stays about the same.

### Why is dictionary lookup faster?

Linear search is **O(n)**. It compares the target id with each record in turn, so on average it checks half the list, and all of it when the id does not exist. A Python dictionary is a **hash table**. The key is hashed to a slot, so the lookup goes straight to the record in **O(1)** average time, whatever the size of the dataset. Building the index costs O(n) once. After that it is much faster for repeated lookups, which is how an API is used. Our store updates the index on every POST, PUT and DELETE.

### Other data structures and algorithms

- **Binary search** on a list sorted by id – O(log n). Uses less memory than a hash table and supports range queries.
- **Balanced BST / B+ tree** – O(log n) lookups with ordered keys. This is how database indexes (e.g. MySQL InnoDB) speed up searches by id or by date range.
- **Secondary hash indexes** (e.g. `transaction_type → [ids]`, `transaction_ref → id`) make `type=PAYMENT` filtering fast instead of scanning all records.
- **Trie** for prefix search on names or phone numbers.

## 4. Reflection: Limitations of Basic Authentication

---

Basic Auth is simple to set up, and that is why we used it here. It is weak for a real financial API:

- **Base64 is encoding, not encryption.** `YWRTaW46cGFzc3dvcmQxMjM=` decodes to `admin:password123` in one step. Over plain HTTP anyone on the network can read the credentials.
- **Credentials are sent with every request.** Each request gives an attacker another chance to capture the real password, and it is often cached by browsers and tools or written to logs.

- **No expiry or revocation.** A leaked password works until someone changes it, and changing it breaks every client.
- **No scopes or roles.** Every authenticated user has full read/write/delete access.
- **No built-in protection against brute force or replay.** There is no lockout, rate limiting or nonce.
- **No proper logout,** and a single shared account makes auditing who did what impossible.

### Stronger alternatives

- **JWT (JSON Web Tokens)** – the user logs in once and receives a signed, short-lived token (e.g. 15 minutes) that carries their identity and roles. The password is not sent again. Tokens can be verified without a database lookup and paired with refresh tokens and a revocation list.
- **OAuth 2.0 / OpenID Connect** – the standard for delegated access. Clients get scoped access tokens (e.g. `transactions:read`) from an authorization server and never see the user's password. Suitable for third-party mobile and web apps.
- **Always use HTTPS/TLS,** whatever the scheme, so credentials and tokens are encrypted in transit.
- **Supporting measures:** hash stored passwords with bcrypt/Argon2, rate limit and lock out after repeated failures, use API keys per client for server-to-server calls, and log all access for auditing.

## 5. Conclusion

---

We parsed the MoMo SMS XML into structured JSON and exposed it through a full CRUD REST API built with plain Python. Every endpoint is protected with Basic Authentication. The API is documented and tested with curl and automated tests. The DSA comparison showed that a dictionary index gives near-constant lookup time while linear search slows down as the data grows. Basic Auth works for this exercise, but a production MoMo API should use HTTPS with JWT or OAuth 2.0.